
ASSIGNMENT 24

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

<https://github.com/omkulawade03/Frontend-Session-Assignments.git>

Import all required libraries

```
train_and_save.py > ...
1  import joblib
2  import pandas as pd
3  import numpy as np
4
5  from sklearn.datasets import load_iris, fetch_california_housing
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8
9  # Classification Imports
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.tree import DecisionTreeClassifier
12 from sklearn.svm import SVC
13 from sklearn.neighbors import KNeighborsClassifier
14 from sklearn.naive_bayes import GaussianNB
15 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
16
17 # Regression Imports
18 from sklearn.linear_model import LinearRegression
19 from sklearn.tree import DecisionTreeRegressor
20 from sklearn.svm import SVR
21 from sklearn.neighbors import KNeighborsRegressor
22 from sklearn.metrics import r2_score
23
```

Q1. (Dataset Selection & Preprocessing)

Choose one classification dataset and one regression dataset (you can use previous datasets or find new ones).

Load and clean both datasets.

Perform necessary preprocessing (encoding, scaling, train-test split).

Clearly identify Independent Features (X) and Dependent Feature (y) for both.

Program Code:

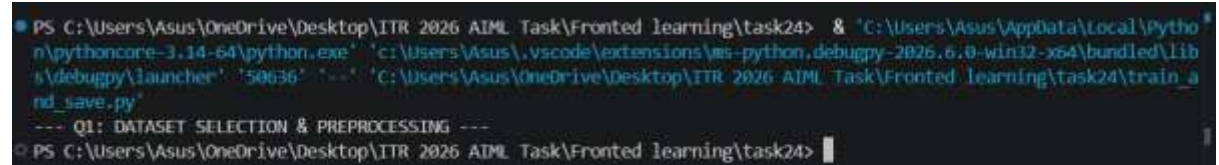
#Questions 01

```

23
24 # =====
25 # Q1: DATASET SELECTION & PREPROCESSING
26 # =====
27 print("--- Q1: DATASET SELECTION & PREPROCESSING ---")
28
29 # --- 1A. Classification Dataset (Iris) ---
30 iris = load_iris(as_frame=True)
31 clf_df = iris.frame
32
33 X_clf = clf_df.drop(columns=['target']) # Independent Features
34 y_clf = clf_df['target']               # Dependent Feature
35
36 # REPLACED: test_score=0.2 -> test_size=0.2
37 X_clf_train, X_clf_test, y_clf_train, y_clf_test = train_test_split(
38 |     X_clf, y_clf, test_size=0.2, random_state=42, stratify=y_clf
39 )
40
41 # Scaling features for distance/gradient based classifiers
42 clf_scaler = StandardScaler()
43 X_clf_train_scaled = clf_scaler.fit_transform(X_clf_train)
44 X_clf_test_scaled = clf_scaler.transform(X_clf_test)
45
46 # --- 1B. Regression Dataset (California Housing - Sampled for Speed) ---
47 california = fetch_california_housing(as_frame=True)
48 reg_df = california.frame.sample(n=3000, random_state=42) # Sampled for faster training
49
50 X_reg = reg_df.drop(columns=['MedHouseVal']) # Independent Features
51 y_reg = reg_df['MedHouseVal']               # Dependent Feature
52
53 # REPLACED: test_score=0.2 -> test_size=0.2
54 X_reg_train, X_reg_test, y_reg_train, y_reg_test = train_test_split(
55 |     X_reg, y_reg, test_size=0.2, random_state=42
56 )
57
58 # Scaling features for regression models
59 reg_scaler = StandardScaler()
60 X_reg_train_scaled = reg_scaler.fit_transform(X_reg_train)
61 X_reg_test_scaled = reg_scaler.transform(X_reg_test)
62
63

```

OUTPUT SCREENSHOT:



A terminal window screenshot showing a command execution. The command is a long string of paths and arguments, including 'python.exe', 'ms-python.debugpy-2026.6.0-win32-x64', and 'train_and_save.py'. The output shows a separator '--- Q1: DATASET SELECTION & PREPROCESSING ---' followed by a new prompt 'PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task24>'.

```
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task24> & "C:\Users\Asus\AppData\Local\Python\pythoncore-3.14-64\python.exe" "c:\Users\Asus\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher" "50636" "---" "C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task24\train_and_save.py"
--- Q1: DATASET SELECTION & PREPROCESSING ---
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task24>
```

Q2. (Classification Algorithms)

On the classification dataset, train and evaluate the following models:

Logistic Regression

Decision Tree Classifier

Support Vector Machine (SVM)

K-Nearest Neighbors (KNN)

Naive Bayes

For each model:

Train the model

Make predictions

Calculate Accuracy, Confusion Matrix, and Classification Report

Create a comparison table of all models' performance.

Program Code:

#Questions 02

```

64 # =====
65 # Q2: CLASSIFICATION ALGORITHMS EVALUATION
66 # =====
67 print("\n--- Q2: CLASSIFICATION EVALUATION ---")
68
69 clf_models = {
70     "Logistic Regression": (LogisticRegression(max_iter=200), True),
71     "Decision Tree Classifier": (DecisionTreeClassifier(random_state=42), False),
72     "Support Vector Machine (SVM)": (SVC(), True),
73     "K-Nearest Neighbors (KNN)": (KNeighborsClassifier(), True),
74     "Naive Bayes": (GaussianNB(), False)
75 }
76
77 clf_results = {}
78 trained_clf_models = {}
79
80 for name, (model, needs_scale) in clf_models.items():
81     X_tr = X_clf_train_scaled if needs_scale else X_clf_train
82     X_te = X_clf_test_scaled if needs_scale else X_clf_test
83
84     # Train & Predict
85     model.fit(X_tr, y_clf_train)
86     y_pred = model.predict(X_te)
87
88     # Evaluate
89     acc = accuracy_score(y_clf_test, y_pred)
90     cm = confusion_matrix(y_clf_test, y_pred)
91     cr = classification_report(y_clf_test, y_pred, target_names=iris.target_names)
92
93     clf_results[name] = acc
94     trained_clf_models[name] = model
95
96     print(f"\n>>> Model: {name}")
97     print(f"Accuracy: {acc:.4f}")
98     print("Confusion Matrix:\n", cm)
99     print("Classification Report:\n", cr)
100
101 # Comparison Table
102 clf_summary = pd.DataFrame(list(clf_results.items()), columns=['Model', 'Accuracy']).sort_values('Accuracy')
103 print("\nClassification Models Comparison:")
104 print(clf_summary.to_string(index=False))
105

```

OUTPUT SCREENSHOT:

--- Q2: CLASSIFICATION EVALUATION ---

>>> Model: Logistic Regression

Accuracy: 0.9333

Confusion Matrix:

[[10 0 0]

[0 9 1]

[0 1 9]]

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

>>> Model: Decision Tree Classifier

Accuracy: 0.9333

Confusion Matrix:

[[10 0 0]

[0 9 1]

[0 1 9]]

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30


```
accuracy          0.93          30
macro avg         0.93          30
weighted avg      0.93          30
```

```
>>> Model: Support Vector Machine (SVM)
```

```
Accuracy: 0.9667
```

```
Confusion Matrix:
```

```
[[10  0  0]
```

```
 [ 0  9  1]
```

```
 [ 0  0 10]]
```

```
Classification Report:
```

```
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00        10
  versicolor      1.00        0.90        0.95        10
   virginica       0.91        1.00        0.95        10

 accuracy          0.97          30
 macro avg         0.97          30
weighted avg       0.97          30
```

```
>>> Model: K-Nearest Neighbors (KNN)
```

```
Accuracy: 0.9333
```

```
Confusion Matrix:
```

```
[[10  0  0]
```

```
 [ 0 10  0]
```

```
 [ 0  2  8]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.83	1.00	0.91	10
virginica	1.00	0.80	0.89	10
accuracy			0.93	30
macro avg	0.94	0.93	0.93	30
weighted avg	0.94	0.93	0.93	30

```
>>> Model: Naive Bayes
```

```
Accuracy: 0.9667
```

```
Confusion Matrix:
```

```
[[10  0  0]
```

```
 [ 0  9  1]
```

```
 [ 0  0 10]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	0.90	0.95	10
virginica	0.91	1.00	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

```
Classification Models Comparison:
```

Model	Accuracy
Support Vector Machine (SVM)	0.966667
Naive Bayes	0.966667
Logistic Regression	0.933333
Decision Tree Classifier	0.933333
K-Nearest Neighbors (KNN)	0.933333

Q3. (Regression Algorithms)

On the regression dataset, train and evaluate the following models:

Linear Regression

Decision Tree Regressor

Support Vector Regressor (SVR)

K-Nearest Neighbors Regressor

For each model:

Train the model

Make predictions

Calculate R2 Score

Create a comparison table showing which model performed best.

Program Code:

#Questions 03

```

107 # =====
108 # Q3: REGRESSION ALGORITHMS EVALUATION
109 # =====
110 print("\n--- Q3: REGRESSION EVALUATION ---")
111
112 reg_models = {
113     "Linear Regression": (LinearRegression(), True),
114     "Decision Tree Regressor": (DecisionTreeRegressor(random_state=42), False),
115     "Support Vector Regressor (SVR)": (SVR(), True),
116     "K-Nearest Neighbors Regressor": (KNeighborsRegressor(), True)
117 }
118
119 reg_results = {}
120 trained_reg_models = {}
121
122 for name, (model, needs_scale) in reg_models.items():
123     X_tr = X_reg_train_scaled if needs_scale else X_reg_train
124     X_te = X_reg_test_scaled if needs_scale else X_reg_test
125
126     # Train & Predict
127     model.fit(X_tr, y_reg_train)
128     y_pred = model.predict(X_te)
129
130     # Evaluate
131     r2 = r2_score(y_reg_test, y_pred)
132     reg_results[name] = r2
133     trained_reg_models[name] = model
134
135 # Comparison Table
136 reg_summary = pd.DataFrame(list(reg_results.items()), columns=['Model', 'R2 Score']).sort_values
137 print("\nRegression Models Comparison:")
138 print(reg_summary.to_string(index=False))
139
140

```

OUTPUT SCREENSHOT:

```
--- Q3: REGRESSION EVALUATION ---  
  
Regression Models Comparison:  
                        Model  R2 Score  
Support Vector Regressor (SVR) 0.736416  
  K-Nearest Neighbors Regressor 0.674146  
            Linear Regression 0.625249  
      Decision Tree Regressor 0.531825  
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task24>
```

Q4. (Best Model Selection & Saving)

From Q2 and Q3:

Select the best classification model and best regression model based on performance metrics.

Save both best models using joblib (also save scaler and columns if used).

Program Code:

#Questions 04

```
141 # =====
142 # Q4: BEST MODEL SELECTION & SAVING
143 # =====
144 print("\n--- Q4: SAVING BEST MODELS ---")
145
146 best_clf_name = clf_summary.iloc[0]['Model']
147 best_reg_name = reg_summary.iloc[0]['Model']
148
149 print(f"Best Classification Model: {best_clf_name} (Accuracy: {clf_results[best_clf_name]:.4f})
150 print(f"Best Regression Model: {best_reg_name} (R2 Score: {reg_results[best_reg_name]:.4f})")
151
152 # Packaging best models and preprocessing metadata
153 best_clf_payload = {
154     "model_name": best_clf_name,
155     "model": trained_clf_models[best_clf_name],
156     "scaler": clf_scaler,
157     "features": list(X_clf.columns),
158     "target_names": list(iris.target_names),
159     "needs_scaling": clf_models[best_clf_name][1]
160 }
161
162 best_reg_payload = {
163     "model_name": best_reg_name,
164     "model": trained_reg_models[best_reg_name],
165     "scaler": reg_scaler,
166     "features": list(X_reg.columns),
167     "needs_scaling": reg_models[best_reg_name][1]
168 }
169
```

```
170 # Packaging all models so the Streamlit application can use any chosen algorithm
171 all_models_payload = {
172     "classification": {
173         "models": trained_clf_models,
174         "scaler": clf_scaler,
175         "features": list(X_clf.columns),
176         "target_names": list(iris.target_names),
177         "scaling_reqs": {name: req for name, (_, req) in clf_models.items()}
178     },
179     "regression": {
180         "models": trained_reg_models,
181         "scaler": reg_scaler,
182         "features": list(X_reg.columns),
183         "scaling_reqs": {name: req for name, (_, req) in reg_models.items()}
184     }
185 }
186
187 joblib.dump(best_clf_payload, 'best_classification_model.pkl')
188 joblib.dump(best_reg_payload, 'best_regression_model.pkl')
189 joblib.dump(all_models_payload, 'all_trained_models.pkl')
190
191 print("Models, Scalers, and Feature maps successfully saved via joblib!")
192
193
```

OUTPUT SCREENSHOT:

Q5. (Streamlit Multi-Model App)

Build a Streamlit web application that allows the user to:

1. Choose the problem type (Classification or Regression)
 2. Select any of the algorithms trained above
 3. Enter input values
 4. Get prediction from the selected model
-

Program Code:

#Questions 05

```
1  import streamlit as st
2  import joblib
3  import pandas as pd
4  import numpy as np
5
6  st.set_page_config(
7      page_title="AIML Multi-Model Inference App",
8      page_icon="🤖",
9      layout="wide"
10 )
11
12 # Load saved models artifact
13 @st.cache_resource
14 def load_artifacts():
15     return joblib.load('all_trained_models.pkl')
16
17 try:
18     artifacts = load_artifacts()
19 except Exception as e:
20     st.error("Could not find 'all_trained_models.pkl'. Please run 'train_and_save.py' first!")
21     st.stop()
22
23 st.title("🤖 Multi-Model ML Prediction App")
24 st.markdown("Select a problem type, choose a model, adjust feature inputs, and get immediate predictions.")
25
26 # Sidebar Selection
27 st.sidebar.header("Configuration")
28 problem_type = st.sidebar.radio("1. Select Problem Type", ["Classification", "Regression"])
29
30 if problem_type == "Classification":
31     clf_data = artifacts["classification"]
32     selected_model_name = st.sidebar.selectbox("2. Select Classification Model", list(clf_data["models"].keys()))
33
34     st.header(f"Iris Flower Classification - {selected_model_name}")
35     st.markdown("### Enter Input Feature Values:")
36
```



```

36
37     col1, col2 = st.columns(2)
38     inputs = {}
39
40     # Feature inputs (Iris defaults)
41     feature_defaults = {
42         "sepal length (cm)": 5.8,
43         "sepal width (cm)": 3.0,
44         "petal length (cm)": 3.8,
45         "petal width (cm)": 1.2
46     }
47
48     for i, feature in enumerate(clf_data["features"]):
49         target_col = col1 if i % 2 == 0 else col2
50         inputs[feature] = target_col.number_input(
51             f"**{feature.title()}**",
52             value=float(feature_defaults.get(feature, 1.0)),
53             step=0.1
54         )
55
56     if st.button("Predict Species", type="primary"):
57         input_df = pd.DataFrame([inputs])
58
59         # Check scaling requirements
60         if clf_data["scaling_reqs"][selected_model_name]:
61             input_processed = clf_data["scaler"].transform(input_df)
62         else:
63             input_processed = input_df
64
65         model = clf_data["models"][selected_model_name]
66         prediction = model.predict(input_processed)[0]
67         predicted_class = clf_data["target_names"][prediction]
68
69         st.success(f"**Predicted Target Class:** {predicted_class.title()} (Class ID: {prediction})")
70
71     if hasattr(model, "predict_proba"):
72         probs = model.predict_proba(input_processed)[0]
73         prob_df = pd.DataFrame([
74             "Species": clf_data["target_names"],
75             "Probability": probs
76         ])
77         st.bar_chart(prob_df.set_index("Species"))
78
79     else:
80         reg_data = artifacts["regression"]
81         selected_model_name = st.sidebar.selectbox("2. Select Regression Model", list(reg_data["models"].key
82
83         st.header(f"California Housing Price Prediction - {selected_model_name}")
84         st.markdown("### Enter Input Feature Values:")
85
86         col1, col2 = st.columns(2)
87         inputs = {}
88
89         # Feature defaults for California Housing
90         feature_defaults = {
91             "MedInc": 3.87,
92             "HouseAge": 28.0,
93             "AveRooms": 5.4,
94             "AveBedrms": 1.1,
95             "Population": 1425.0,
96             "AveOccup": 3.0,
97             "Latitude": 35.6,
98             "Longitude": -119.5
99         }
100

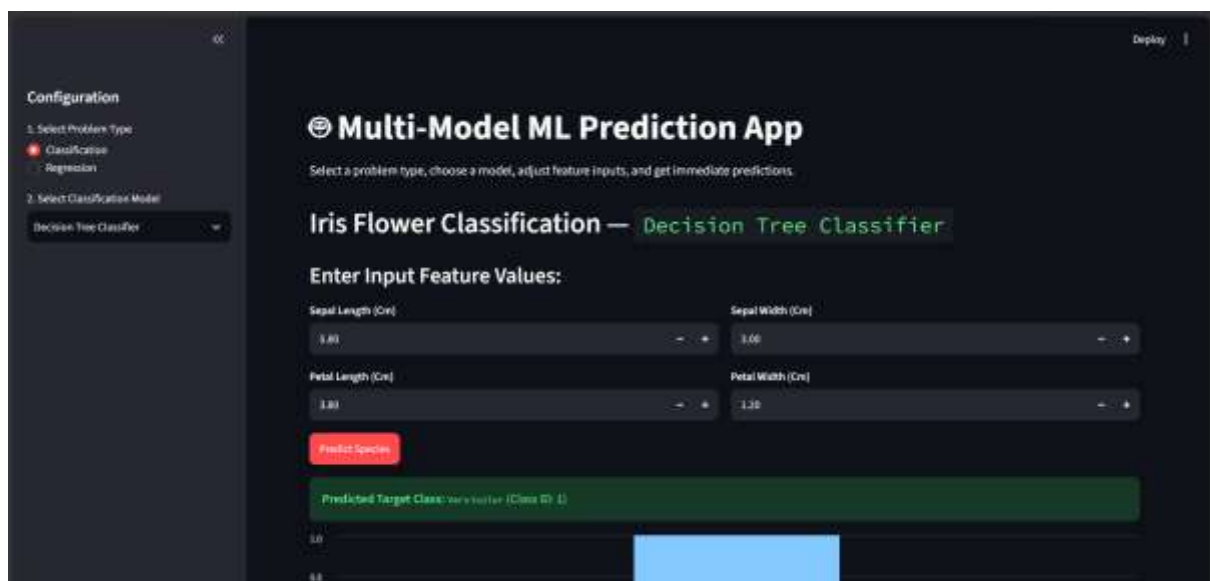
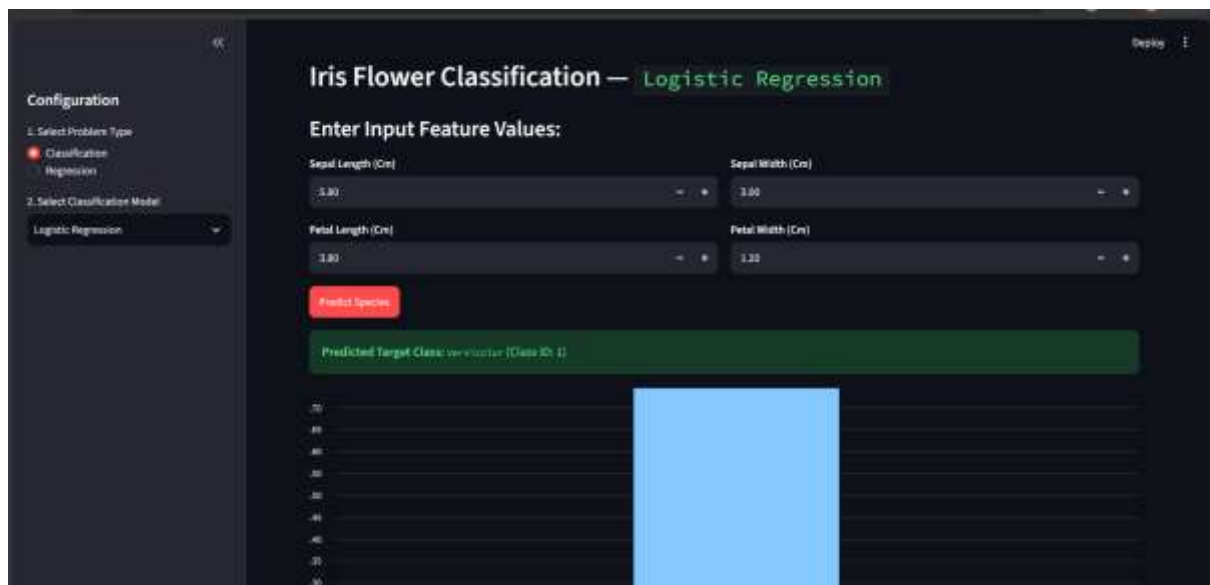
```

```

100
101     feature in enumerate(reg_data["features"]):
102         col = col1 if i % 2 == 0 else col2
103         ts[feature] = target_col.number_input(
104             f"***{feature}***",
105             value=float(feature_defaults.get(feature, 1.0)),
106             step=0.1
107         )
108
109     button("Predict Median House Value", type="primary"):
110         t_df = pd.DataFrame([inputs])
111
112         # check scaling requirements
113         reg_data["scaling_reqs"][selected_model_name]:
114             input_processed = reg_data["scaler"].transform(input_df)
115         :
116             input_processed = input_df
117
118         l = reg_data["models"][selected_model_name]
119         prediction = model.predict(input_processed)[0]
120
121         # get variable MedHouseVal is in $100,000s
122         al_value = prediction * 100000
123
124         access(f"***Predicted Median House Value:** ${actual_value:,.2f}" (f"prediction: {prediction:,.4f}" in $100k units)"))

```

OUTPUT SCREENSHOT:





Configuration

1. Select Problem Type

- Classification
- Regression

2. Select Classification Model

Support Vector Machine (SVM)

Deploy

Multi-Model ML Prediction App

Select a problem type, choose a model, adjust feature inputs, and get immediate predictions.

Iris Flower Classification — Support Vector Machine (SVM)

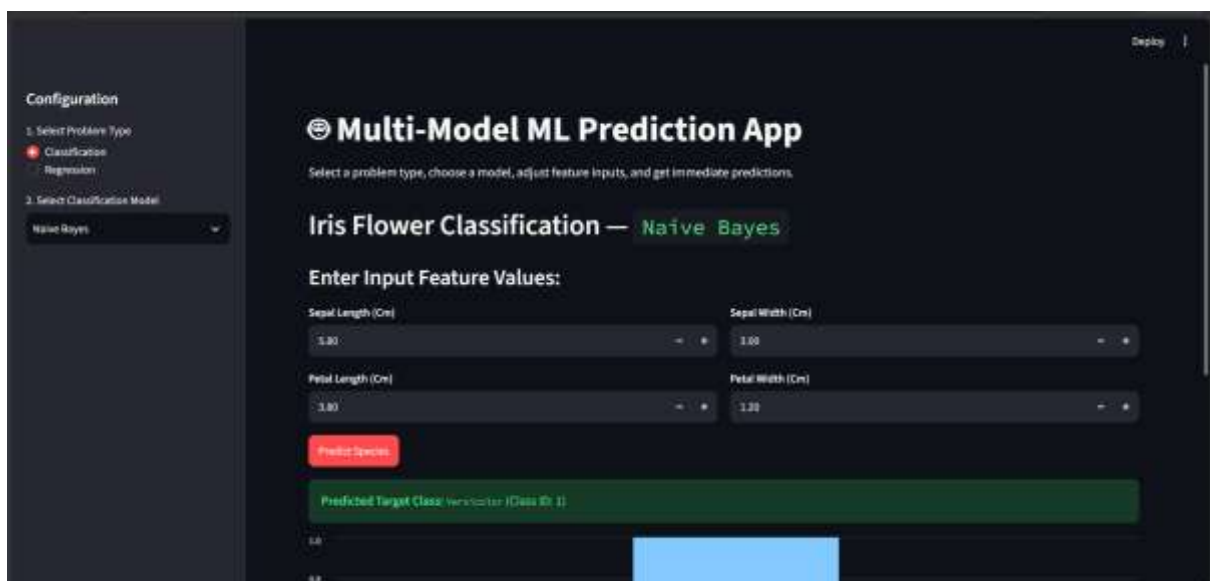
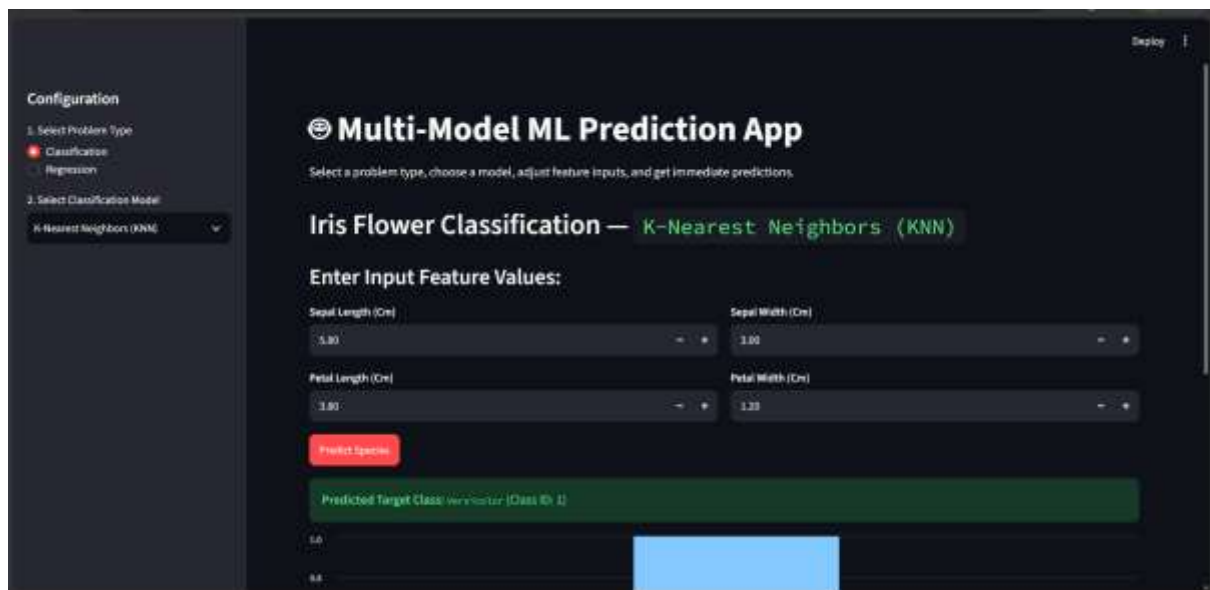
Enter Input Feature Values:

Sepal Length (Cm): 5.80 Sepal Width (Cm): 3.80

Petal Length (Cm): 3.80 Petal Width (Cm): 3.20

Predict Species

Predicted Target Class: versicolor (Class ID: 1)





Configuration

1. Select Problem Type

- Classification
- Regression**

2. Select Regression Model

- Linear Regression**

California Housing Price Prediction — Linear Regression

Enter Input Feature Values:

MedInc	3.87	HouseAge	20.00
AveRooms	5.49	AveBedrms	1.00
Population	1425.00	AveOccup	3.00
Latitude	35.61	Longitude	-122.50

Predict Median House Value

Predicted Median House Value: 5166.479, 25 (2, 000) in \$100k units

66

Deploy

Configuration

1. Select Problem Type

Classification

Regression

2. Select Regression Model

Decision Tree Regressor

California Housing Price Prediction — Decision Tree Regressor

Enter Input Feature Values:

MedInc	3.87	HouseAge	28.00
AveRooms	5.40	AveBedrms	3.33
Population	1425.00	AveOccup	3.00
Latitude	35.40	Longitude	-119.50

Predict Median House Value

Predicted Median House Value: 166,396.46 (\$1,663,964.60 in \$100k units)

66

Deploy

Configuration

1. Select Problem Type

Classification

Regression

2. Select Regression Model

Support Vector Regressor (SVR)

California Housing Price Prediction — Support Vector Regressor (SVR)

Enter Input Feature Values:

MedInc	3.87	HouseAge	28.00
AveRooms	5.40	AveBedrms	3.33
Population	1425.00	AveOccup	3.00
Latitude	35.40	Longitude	-119.50

Predict Median House Value

Predicted Median House Value: 1364,401.97 (\$13,644,019.70 in \$100k units)

66

Deploy

Configuration

1. Select Problem Type

Classification

Regression

2. Select Regression Model

K-Nearest Neighbors Regressor

California Housing Price Prediction — K-Nearest Neighbors Regressor

Enter Input Feature Values:

MedInc	3.87	HouseAge	28.00
AveRooms	5.40	AveBedrms	3.33
Population	1425.00	AveOccup	3.00
Latitude	35.40	Longitude	-119.50

Predict Median House Value

Predicted Median House Value: 166,396.46 (\$1,663,964.60 in \$100k units)

